

Guia 1

Relatório da Evolução da Sprint3

Equipe

Anna Cecilia Guimarães Maiques Lima de Carvalho -RM: 570955

Caio Eguia Ceschini -RM: 573847

Gabriel Henrique S. de Melo Rodrigues - RM: 573093

Fernando Bonfim Hoefle - RM: 569920

Arthur de Oliveira Carvalho - RM: 573499

Turma 1CCA

Resumo da evolução

Nas Sprints 1 e 2, o GoodCharge utilizava chamadas diretas à API da OpenAI, com um system prompt e histórico da conversa controlado manualmente.

Na Sprint 03, o projeto foi refatorado para utilizar o **OpenAI Agents SDK**, passando a utilizar Agent, Runner e SQLiteSession. Também foram adicionadas ferramentas (Function Tools) para consultar consumo, cobrança, carregadores, falhas e realizar reservas. Além disso, foram incluídos testes de segurança e comparação entre dois modelos de linguagem. Assim, o chatbot passou de uma estrutura com chamadas manuais para uma arquitetura com agente, memória e ferramentas integradas.

Alterações no Projeto

O framework escolhido foi o **OpenAI Agents SDK**, pois permite organizar o agente, suas instruções, ferramentas e memória em uma mesma estrutura.

Os principais componentes utilizados foram:

- **Agent:** define o comportamento e as instruções do GoodCharge.
- **Runner:** executa as interações com o agente.
- **SQLiteSession:** mantém a memória da conversa por sessão.
- **Function Tools:** permitem que o agente consulte os dados simulados e execute ações, como reservas.

A principal vantagem foi deixar as funções do sistema mais organizadas e permitir que o agente use as ferramentas quando necessário.

Como trade-off, a nova arquitetura ficou mais complexa que a anterior, pois passou a exigir a configuração de agente, sessão de memória e ferramentas.

Comparativo Sprint 1 e 2 × Sprint3

O comparativo entre as versões é:

Arquitetura

- Sprints 1 e 2: chamadas manuais à LLM.
- Sprint 03: arquitetura utilizando agente.

Framework

- Sprints 1 e 2: não utilizava framework de agentes.
- Sprint 03: OpenAI Agents SDK.

Memória

- Sprints 1 e 2: histórico controlado manualmente.
- Sprint 03: memória utilizando SQLiteSession.

Ferramentas

- Sprints 1 e 2: não utilizava Function Tools.
- Sprint 03: Function Tools para consumo, cobrança, carregadores, falhas e reservas.

Reservas

- Sprints 1 e 2: fluxo mais limitado.
- Sprint 03: reserva utilizando o número do carregador e o horário informado pelo usuário.

Modelos

- Sprints 1 e 2: um modelo utilizado.
- Sprint 03: dois modelos testados.

Segurança

- Sprints 1 e 2: regras no system prompt.
- Sprint 03: regras de comportamento e testes de segurança, incluindo Prompt Injection.

Comparação dos modelos

Foram realizados 7 testes para cada modelo.

gpt-4o-mini

- Tokens de entrada por média: 1.136,43
- Tokens de saída por média: 45,71
- Tokens totais por média: 1.182,14
- Latência média: 2,42 segundos

gpt-5-nano

- Tokens de entrada por média: 1.459,71
- Tokens de saída por média: 810,14
- Tokens totais por média: 2.269,86
- Latência média: 5,78 segundos

Nos testes realizados, o gpt-4o-mini apresentou menor quantidade média de tokens e menor latência. Os dois modelos recusaram o pedido de revelar o system prompt no teste de Prompt Injection.

Com base nos resultados obtidos, o **gpt-4o-mini foi utilizado como modelo final**, principalmente pelos menores valores de tokens e latência observados nos testes.

Problemas encontrados e soluções

Problema 1 — Memória da conversa

Problema: nas versões anteriores, o histórico era controlado manualmente, deixando o gerenciamento da memória menos organizado.

Possíveis alternativas: continuar utilizando uma lista manual de histórico ou utilizar o recurso de memória do framework.

Solução adotada: utilização do SQLite Session para armazenar a memória por sessão.

Justificativa: permite que o agente recupere informações apresentadas anteriormente pelo usuário e atende ao requisito de memória conversacional.

Problema 2 — Reserva de carregadores

Problema: o fluxo anterior de reserva era mais limitado.

Possíveis alternativas: manter uma reserva fixa ou criar uma ferramenta específica para controlar as reservas.

Solução adotada: criação da ferramenta reservar_carregador(numero, horario).

Justificativa: permite receber o número do carregador e o horário informado pelo usuário e verificar os dados antes de confirmar a reserva.

Problema 3 — Controle das respostas

Problema: o chatbot poderia responder informações que não estavam disponíveis nos dados simulados ou sair do contexto do projeto.

Possíveis alternativas: deixar o comportamento apenas por conta do modelo ou reforçar as regras e realizar testes específicos.

Solução adotada: foram adicionadas regras para impedir a invenção de especificações técnicas, limitar orientações jurídicas e financeiras e impedir orientações elétricas potencialmente perigosas. Também foram realizados testes de segurança, incluindo Prompt Injection.

Justificativa: as regras ajudam a manter o agente dentro do objetivo do GoodCharge e deixam suas respostas mais controladas.

Divisão da equipe

Anna Cecilia Guimarães Maiques Lima de Carvalho: desenvolvimento e documentação.

Caio Eguia Ceschini: Desenvolvimento.

Gabriel Henrique S. de Melo Rodrigues: Desenvolvimento.

Fernando Bonfim Hoefle: Desenvolvimento.

Arthur de Oliveira Carvalho: desenvolvimento.

Guia 3

